SkillGap

From Resume to Roadmap — Instantly.

AI-Adaptive Onboarding Engine
Parses resumes and job descriptions to identify skill gaps
and builds personalised, dependency-aware learning pathways.

Submitted to: ARTPARK CodeForge Hackathon 2026
Team: Team HackOS
Members: Guruprasad Shinde | Harshvardhan Sathe | Dhanvantri Panjwani
Repository: github.com/Guru2025-KIT/SkillGap

135	18	37	<3s
Skills	Categories	Course Links	Analysis Time

Table of Contents

Document Scope

This document covers the problem understanding, system architecture, technology stack, core algorithms, key features, datasets, and validation metrics for SkillGap — an AI-Adaptive Onboarding Engine submitted to the ARTPARK CodeForge Hackathon 2026.

1. Problem Understanding

Corporate onboarding today relies on static, one-size-fits-all training programs. The same set of slides and modules is delivered to every new hire regardless of their existing skill set, experience level, or the specific requirements of their role. This creates two costly and opposing failure modes that organisations consistently underestimate.

Over-training experienced hires	Senior engineers and specialists are forced to sit through foundational content they mastered years ago. Engagement drops from day one, and skilled hires quickly form the impression that the organisation does not value their existing expertise.
Under-supporting beginners	Junior employees are dropped into advanced modules without the scaffolding needed to understand them. Confidence takes a hit before they have shipped a single feature, leading to early attrition and extended ramp times.
Zero visibility for HR and managers	No structured mechanism exists to assess what a new hire actually knows before their first day. Managers discover skill gaps only when something fails on the job — by which point costly mistakes have already been made.

1.1 Our Approach

SkillGap addresses all three failure modes with a single, transparent pipeline. A new hire uploads their resume and a job description. In under three seconds, the system:

- Extracts skills from both documents using NLP alias matching across 135 canonical skills
- Infers proficiency levels from years of experience and contextual keywords
- Computes a gap score (0–10) for every skill the JD requires that the resume under-delivers
- Builds a dependency-aware learning pathway ordered by prerequisites and priority
- Provides a full reasoning trace so every recommendation is auditable

The result is a personalised roadmap — not generic training — built entirely from the candidate's actual skills versus the actual role requirements. No black box. No hallucinated courses. Every decision explained.

2. System Architecture

SkillGap follows a clean client-server architecture with a React 18 single-page application on the frontend and a FastAPI backend on Python 3.11. All services are containerised with Docker and orchestrated with Docker Compose for one-command deployment.

2.1 High-Level Data Flow

Step	Stage	Service	Description
01	PARSE	parser.py	Extract raw text from uploaded PDF, DOCX, or TXT files
02	EXTRACT	extractor.py	Alias-based NLP matching across 135-skill taxonomy; infer proficienc
03	SCORE	extractor.py	Compute gap_score = level_delta x 2.5 x importance_weight for each skill
04	PATH	pathway.py	Build NetworkX DiGraph; topological sort; bin modules into 4 phases
05	EXTRAS	extras.py	ATS simulation, resume tips, weekly roadmap, job matching
06	VISUALISE	React (10 tabs)	Render interactive dashboard with charts, Gantt, and apply links

2.2 Backend Architecture

The backend exposes four REST endpoints under FastAPI. All endpoints accept multipart/form-data and return JSON. The core pipeline (steps 1–4) is entirely deterministic — no external API calls, no model inference at runtime.

`GET /health`	Returns service status. Used by Docker healthcheck every 30 seconds.
`POST /analyze`	Accepts resume + JD files. Runs the full 6-stage pipeline. Returns gaps, pathway, ATS score, resume tips, roadmap, and job recommendations in a single JSON response.
`POST /analyze-text`	Same as /analyze but accepts raw text strings. Useful for testing via Swagger UI at /docs.
`POST /interview-prep`	Accepts candidate name, role, gaps (JSON), strengths (JSON), readiness score. Returns a rule-based personalised interview coaching guide.

2.3 Frontend Architecture

The frontend is a React 18 single-page application using React Router v6. All analysis state is held in App.js and passed as props — no external state management library is needed. Recharts handles all data visualisations. The design system is built entirely on CSS custom properties with no third-party framework.

Tab	Description
■ Pathway	Phased learning roadmap with module cards, verified course links, and estimated hours
■ Gaps	Scored gap table, horizontal bar chart (top 10), category breakdown pie chart
■ Skills	Resume skills with proficiency bars, skill radar chart, coverage by category
■ Timeline	Gantt chart — week-by-week view of the full learning plan
■ Interview	Rule-based personalised interview coaching — technical and behavioural questions
■ ATS Score	5-dimension ATS simulation scoring keyword match, sections, quantification, verbs, leng
✍ Resume Tips	Prioritised fix cards (high/medium/low) with specific actionable instructions
■ Roadmap	Week-by-week learning calendar with daily targets and phase milestone banners
■ Jobs	Role-fit matching against 12 profiles with LinkedIn, Naukri, Indeed apply buttons
■ Reasoning	Full audit trail — gap formula, ATS breakdown, job matching rationale

3. Technology Stack

3.1 Backend Libraries

Library	Version	Purpose
FastAPI	0.111.0	Async REST API framework with automatic Swagger UI
Uvicorn	0.30.1	ASGI production server
spaCy en_core_web_sm	3.7.5	English NLP: tokenisation and POS tagging
NetworkX	3.3	Directed dependency graph and topological sort
scikit-learn	1.5.1	TF-IDF skill weighting
NumPy	1.26.4	Level index arithmetic for gap scoring
Pandas	2.2.2	Taxonomy data manipulation
PyPDF2	3.0.1	Resume PDF text extraction
docx2txt	0.8	Word document text extraction
Pydantic	2.7.4	Request and response schema validation

3.2 Frontend Libraries

Library	Version	Purpose
React	18.3.1	Component-based single-page application
React Router DOM	6.24.1	Client-side navigation
Recharts	2.12.7	Radar, Bar, Pie, and Gantt visualisations
Axios	1.7.2	HTTP client with timeout handling
react-dropzone	14.2.3	Drag-and-drop file upload UX

3.3 Infrastructure

Tool	Purpose
Docker + Docker Compose	One-command reproducible deployment of both services
nginx Alpine	Static file serving and React SPA routing
Python 3.11-slim	Minimal, production-ready backend container image
Node 20 Alpine	Multi-stage frontend build image

No LLM in Core Pipeline	LLMs hallucinate. For an onboarding engine, hallucinated course links or wrong skill assessments have real consequences. SkillGap's extraction, scoring, and pathing are 100% deterministic — the same resume always produces the same pathway. There is no model to prompt-inject, no API rate limit, and no unpredictable runtime behaviour.

4. Skill-Gap Analysis Logic

All logic lives in backend/services/extractor.py. No language model is used at any stage — every result is deterministic and fully reproducible.

4.1 Text Normalisation

Input text is normalised to lowercase with special characters stripped, except for characters meaningful in skill names: +, #, . and /. This ensures C++, C#, and Node.js survive the cleaning step intact.

```
def normalize(text): text = text.lower() text = re.sub(r'[^\w\s\.+#/]', ' ', text) # keep +, #, ., / text = re.sub(r'\s+', ' ', text) return text.strip()
```

4.2 Alias Matching

The taxonomy defines 135 canonical skill keys with 400+ aliases. Aliases are sorted by length descending before matching so that "react native" always matches before "react". Word boundaries prevent "go" from matching inside "django" or "postgres".

4.3 Proficiency Inference

Two independent signals are cross-checked:

Signal	Method	Window
Years of experience	Regex patterns: "3 years of Python", "Python (5+ yrs)"	±60 chars
Context keywords	Keyword lookup: "expert", "lead", "architect", "exposure to"	±80 chars

4.4 Gap Scoring Formula

Each gap is scored on a 0–10 scale using:

gap_score = max(0, req_idx - curr_idx) x 2.5 x importance_weight LEVEL_ORDER: none=0, beginner=1, intermediate=2, advanced=3, expert=4 importance_weight: critical = 1.00 (required, must-have, essential, mandatory) important = 0.75 (preferred, expected, strong)
nice-to-have = 0.45 (a plus, bonus, optional) Maximum score = 10.0

Worked Example	Candidate has Python at beginner level. JD requires Python at advanced, marked critical. gap_score = (3 - 1) x 2.5 x 1.0 = 5.0 / 10 => Phase 2: Skill Consolidation => Module: Intermediate Python (12 hrs)

4.5 Readiness Score

```
readiness = 100 - (actual_penalty / max_possible_penalty) x 100 actual_penalty = sum(importance_weight x gap_score for all gaps)
max_possible = total_required_skills x 2.0
x 10 Result clamped to [5, 95] -- never claims perfect or total failure
```

5. Adaptive Pathing Algorithm

Implemented in backend/services/pathway.py using NetworkX. This is entirely original logic — no pre-built recommendation engine was used.

5.1 Dependency Graph

A NetworkX DiGraph is constructed where an edge from A to B means "A must be learned before B". Prerequisites are recursively expanded so that transitive dependencies are always respected.

```
G = nx.DiGraph() # Edge A --> B = 'learn A before B' # Examples: javascript --> react --> next.js # python --> machine learning --> pytorch for skill in gap_skills: for prereq in SKILLS[skill]['prerequisites']: G.add_edge(prereq, skill) # recursive expansion
```

5.2 Topological Sort

```
order = list(nx.topological_sort(G)) # Guarantee: javascript always before react # react always before next.js # Cycle detection: falls back to gap-score ordering
```

5.3 Phase Assignment

Phas	Name	Criteria
1	Core Foundations	importance == critical AND gap_score >= 6
2	Skill Consolidation	importance in (critical, important) AND gap_score >= 3
3	Role Proficiency	importance == important AND gap_score < 6
4	Advanced Mastery	Catch-all for nice-to-have skills

Within each phase, modules are sorted by gap_score descending — highest urgency is always addressed first.

5.4 Duration Estimation

```
weeks = max(1, round(phase_hours / 10)) # Assumes 10 hours/week of training capacity # Matches typical new-hire ramp schedule
```

5.5 Course Catalog Lookup

Each module is matched to the best entry in skill_taxonomy.json:

- Priority 1: exact from_level match to the candidate's current level
- Priority 2: closest target_level to the required level
- Priority 3: falls back to a Google search URL — the pathway never returns a null result

6. Key Features

Intelligent Document Parsing	Supports PDF, DOCX, and TXT file formats for both resume and job description. Text is extracted using PyPDF2 (PDF), docx2txt (Word), and UTF-8 decode (plain text). The extraction is robust — image-only PDFs return a clear error message rather than silent failure.
135-Skill NLP Taxonomy	The core knowledge base defines 135 canonical skill keys across 18 job categories with 400+ aliases. Skills span programming, frontend, backend, devops, cloud, data science, ML, security, architecture, management, and emerging technologies including LLMs and blockchain.
Transparent Scoring	Every skill gap receives a score from 0 to 10 based on the level delta and importance weight formula. The score drives both phase assignment and module ordering. No score is generated by a model — it is computed arithmetically and fully auditable.
Dependency-Aware Pathway	The learning pathway respects prerequisite chains. If a candidate needs both React and Next.js, JavaScript is always placed first, then React, then Next.js — regardless of the order they appear in the gap list.
Reasoning Trace	Every /analyze response includes a reasoning_trace object with three plain-English sections explaining the gap analysis method, the topological sort logic, and the phase assignment rationale. This is also rendered as a dedicated tab in the UI.
Zero Hallucinations	All 37 course resources in the catalog are real, manually verified URLs pointing to freeCodeCamp, Coursera, YouTube tutorials, Google courses, and DeepLearning.AI. No course is generated — it is either in the catalog or the system falls back to a Google search link.

7. Extra Features

Four additional features are implemented in backend/services/extras.py. All are deterministic — no external API calls are made.

ATS Score Simulation	Simulates how an Applicant Tracking System scores the resume before a human reads it. Scored across 5 dimensions: keyword match (35 pts), section presence (25 pts), quantified achievements (20 pts), action verb usage (10 pts), and length/density (10 pts). Produces a letter grade A–F and specific tips to improve the score.
Resume Improvement Suggestions	Generates prioritised fix cards (high/medium/low) based on the gap analysis. Each card identifies a specific problem, explains why it matters, and gives a concrete action step. Examples: missing critical JD keywords, underlevelled skills, no summary section, low quantification count, weak action verb usage.
Week-by-Week Learning Roadmap	Converts the phase-based pathway into a day-by-day calendar. Distributes modules across weeks within each phase, calculates daily hour targets, generates week-specific tasks (start docs, build project, update resume), and places milestone banners at phase ends.
Job Recommendations	Matches resume skills against 12 curated role profiles using: fit% = (matched_required / total_required) x 70 + (matched_bonus / total_bonus) x 30. Each role card shows fit percentage, salary range, matched skills, missing required skills, and direct apply buttons for LinkedIn, Naukri, and Indeed.

8. Compliance

8.1 Taxonomy Coverage

Stat	Value
Total canonical skills	135
Job categories	18
Skill aliases	400+
Verified course links	37
Course providers	freeCodeCamp, Coursera, Google, DeepLearning.AI, YouTube

8.2 Compliance Statements

- All datasets are publicly available — no proprietary data was used at any stage
- spaCy en_core_web_sm is used under its MIT licence
- O*NET data is public domain (published by the US Department of Labor)
- No user data is stored, logged, or transmitted beyond the single analysis request
- The core adaptive logic (gap scoring, pathway generation, ATS simulation, job matching) is entirely original implementation — not derived from any pre-built recommendation library
- The system contains no personally identifiable information in any stored form

Repository github.com/Guru2025-KIT/SkillGap
Source code, README, Dockerfiles, skill taxonomy, and 5-slide presentation are all available in the public